Exploring Perceptron for Diabetes Prediction: A Study on Model Optimization

Rubaiyat Alam Ruhin
The University of Adelaide
Adelaide SA 5005
a1930497@adelaide.edu.au

Abstract

This research explores the optimization and behavior of a Single Layer Perceptron (SLP) for diabetes prediction using the PIMA Indians Diabetes dataset [2]. The objective is to gain insights into the model's performance through multiple iterations of hyper-parameter tuning, including adjustments to learning rate, batch size, epochs, and early stopping criteria. Rather than prioritizing high accuracy, the study emphasizes the interpret-ability of model behavior across various configurations. Key performance metrics such as accuracy, precision, recall, and F1-score were assessed to evaluate the impact of each optimization strategy. The analysis demonstrates how incremental adjustments influence the model's learning dynamics and generalization capabilities. This work provides valuable insights into the practical aspects of neural network tuning for medical diagnosis tasks, with a focus on understanding the trade-offs between different hyper-parameter choices.

1. Introduction

Diabetes is a chronic condition affecting millions worldwide [6], and early diagnosis plays a critical role in managing and mitigating its long-term complications . Effective categorization of diabetes-related data is essential for identifying the condition, making accurate diagnoses, and designing treatment plans . In the Fig. 1 we can see, over recent years, the application of machine learning methods has shown promise in analyzing medical data, particularly in the classification of diabetes. Models such as Random Forest, Logistic Regression, and Artificial Neural Networks have been widely explored, yielding various levels of accuracy. For instance, Darolia and Chhillar achieved accuracy of 76.1%, 74.75%, and 77.1% [3] using Random Forest, Logistic Regression, and a Multi-layer Perceptron, respectively . Similarly, Patra and Khuntia employed a Modified K-Nearest Neighbour classifier, reaching an accuracy of 83% [8] , while Hama Saeed utilized techniques such as Extra Tree and Decision Tree classifiers, achieving accuracy

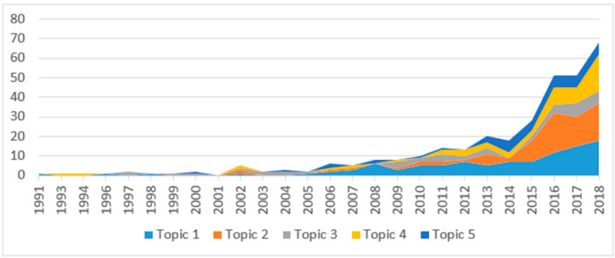


Figure 1. Changes in applications of Artificial Intelligence to diabetes research during 1991–2018 [9].

levels of up to 89% [5] .

Building on these foundational studies, this paper introduces an approach using a Single Layer Perceptron (SLP) to classify diabetes data, leveraging the simplicity and interpret-ability of neural networks. The data that I have worked with is the original PIMA Indians Diabetes dataset [2] instead of the pre-processed one to control data leakage. We will observe the changes in accuracy and performance metrics at different stages of the machine learning workflow. By analyzing these variations, we will aim to understand how hyper-parameter adjustments, such as learning rate, batch size, epochs, and early stopping, affect the Single Layer Perceptron (SLP) model's performance. This will provide insights into the factors driving improvements or declines, helping us better understand simpler neural network architectures for medical data classification.

2. Method

In this study, we followed a systematic workflow for building and optimizing a Single Layer Perceptron (SLP) model for diabetes prediction. The process begins with data acquisition and pre-processing, including exploratory data analysis, cleaning, and scaling. After splitting the data into training, validation, and test sets, the baseline model was initialized and trained. We performed hyper-parameter optimization by adjusting learning rates, batch sizes, epochs, and early stopping criteria to improve performance. Finally,

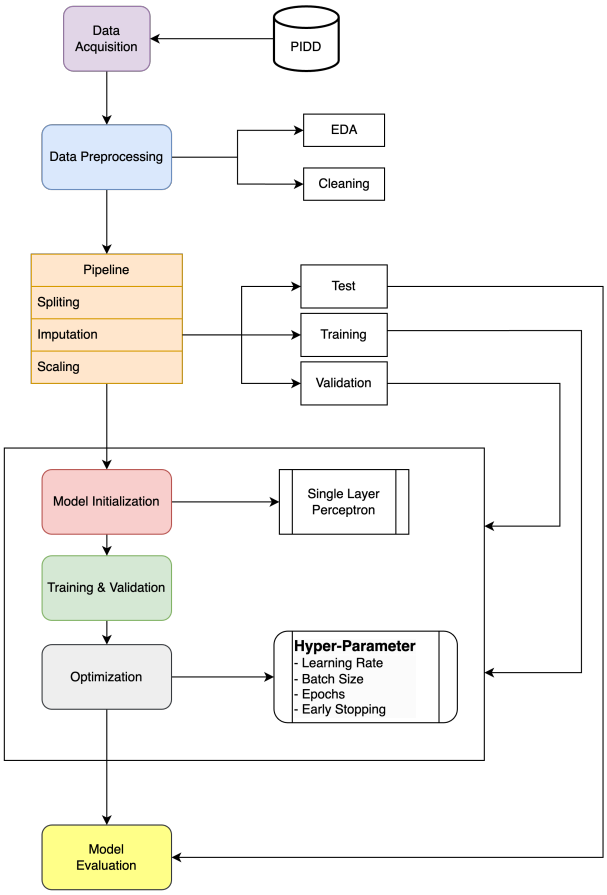


Figure 2. Workflow of the Deep Learning Pipeline.

the model was evaluated on the test set to assess its generalization ability. From the Fig. 2, we can see how each step is interlinked, showing the progression from data acquisition to final evaluation through a structured pipeline.

2.1. Dataset and Pre-processing

The dataset used for this study was the PIMA Indians Diabetes dataset [2], which consists of 768 instances and 8 clinical features, including glucose levels, blood pressure, BMI, and age. Before applying the model, we conducted Exploratory Data Analysis (EDA) and made visualization such as Fig. 3 to identify missing values, outliers, and patterns in the data.

After identifying the missing or unrealistic values, we imputed the missing data using the Multiple Imputation by Chained Equations (MICE) method. Features such as Glucose and Blood Pressure with zero values, which were biologically implausible, were treated as missing and then imputed.

Once the data was cleaned, we split it into training (80%) and test sets (20%), while maintaining the class distribution using stratification. The training set was further split into training and validation sets, again using stratification to conserve the classification ratio. The final shapes of the dataset were:

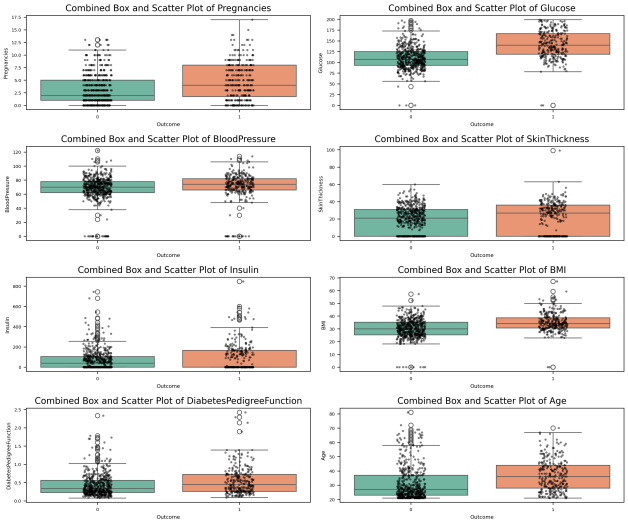


Figure 3. Box-plot of the Original Data to Identify the Outliers.

- Training set shape: (460, 8)
- Validation set shape: (154, 8)
- Test set shape: (154, 8)

Following this, we scaled the dataset using StandardScaler to normalize feature distributions, ensuring all features had a mean of 0 and a standard deviation of 1.

2.2. Baseline Model

The baseline model used for diabetes prediction is a Single Layer Perceptron (SLP). The SLP is one of the simplest forms of artificial neural networks, comprising a single neuron that uses a Sigmoid activation function for binary classification tasks. From the Fig. 4 we can see the simple design of this model. Given the straightforward nature of the SLP, it serves as an ideal model for understanding the impact of different optimization strategies and evaluating the fundamental behavior of neural networks on medical data.

2.2.1 Single Layer Perceptron Model Design

The Perceptron model was built using TensorFlow's Sequential API, consisting of a single Dense layer with one neuron for binary classification. The weighted sum is calculated as [4]:

$$z = \mathbf{w}^T \cdot \mathbf{x} = \sum_{i=1}^{n} w_i x_i \tag{1}$$

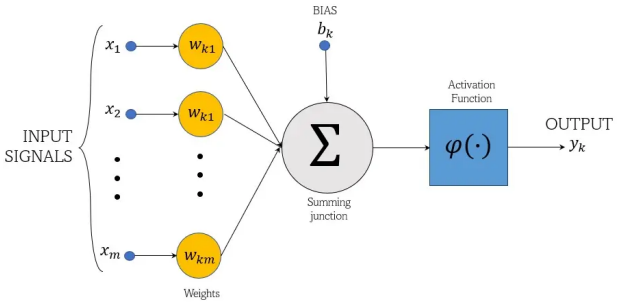


Figure 4. The Architecture of a Single Layer Perceptron [1].

where $\mathbf{w}$ is the weight vector, $\mathbf{x}$ is the input vector, and z is the weighted sum. This is shown in Equation (1).

The activation function used in the Perceptron model can either be a Heaviside step function or a Sigmoid activation function. The Heaviside step function, which outputs binary values, is given by:

$$step(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases} \tag{2}$$

This step function outputs a value of 0 or 1, depending on the input value of z, as shown in Equation (2).

Alternatively, a Sigmoid activation function can be used to output a probability score:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}} \tag{3}$$

In Equation (3), $\hat{y}$ is the predicted probability. The model uses binary cross-entropy as the loss function:

$$\mathcal{L} = -\frac{1}{m} \sum_{j=1}^{m} [y_j \log(\hat{y}_j) + (1 - y_j) \log(1 - \hat{y}_j)] \tag{4}$$

Equation (4) represents the binary cross-entropy loss, where y_j and $\hat{y}_j$ are the true and predicted labels. The model is trained using the Adam optimizer with gradient descent:

$$w_i^{(t+1)} = w_i^{(t)} - \alpha \frac{\partial \mathcal{L}}{\partial w_i} \tag{5}$$

$$b^{(t+1)} = b^{(t)} - \alpha \frac{\partial \mathcal{L}}{\partial b} \tag{6}$$

As shown in Equations (5) and (6), the weights and bias are updated using the gradients of the loss function. Early stopping was applied to prevent over-fitting, with a patience of 10 epochs.

2.2.2 Training and Evaluation

The Perceptron model was trained using the pre-processed training data with a maximum of 100 epochs, a batch size of 32, and early stopping to prevent over-fitting. The early stopping callback monitored the validation loss and halted training if no improvement was observed after a certain number of epochs. The model was evaluated on the validation set, achieving a validation accuracy of 0.7792.

2.3. Hyper-parameter Optimization

To improve the performance of the baseline model, we conducted manual hyper-parameter optimization by exploring various key parameters. The following hyper-parameters were tested:

- Learning Rates: [0.1, 0.01, 0.001, 0.0001]
- Batch Sizes: [16, 32, 64, 128]
- Epochs: [50, 100, 150, 200]
- Early Stopping Patience: [5, 10, 15, 20]

Each parameter was adjusted in isolation while keeping the others fixed to observe its impact on the model's performance. The goal was to fine-tune the model to better generalize to unseen data while preventing over-fitting. After several rounds of experimentation, the best result was achieved with a validation accuracy of 0.8312, using the parameter: Early Stopping Patience set to 5.

2.4. Optimized Model Evaluation

After selecting the best hyper-parameters through validation, the model was retrained using the entire training set with the best patience value of 5. Once retrained, the model was evaluated on the test set to assess its performance on unseen data. Surprisingly, the test accuracy dropped to 0.7078, indicating a decrease in performance compared to the validation accuracy.

3. Experimental Analysis

As the experiments conducted in this study was aimed to evaluate the performance of a Single Layer Perceptron (SLP) for diabetes prediction using the PIMA Indians Diabetes dataset [2]. Following are the observations that I made:

3.1. Baseline Model Result

The baseline model, trained with default hyper-parameters, achieved a validation accuracy of 0.7792. The key performance metrics for the baseline model are shown in Tab. 1. These results indicate that the model had strong

precision and AUC, suggesting that it was effective at correctly identifying non-diabetic cases. However, the relatively lower recall and F1 score suggest that the model struggled to correctly identify all positive cases, potentially missing some diabetic patients. The confusion matrix in Fig. 5 also highlights these misclassifications.

Metric	Score
Validation Accuracy	0.7792
Balanced Accuracy	0.7320
Recall	0.5741
Precision	0.7381
AUC	0.8656
F1 Score	0.6458
F-beta Score (beta=0.1)	0.7360
F-beta Score (beta=10)	0.5753

Table 1. Performance Metrics of The Baseline Model.

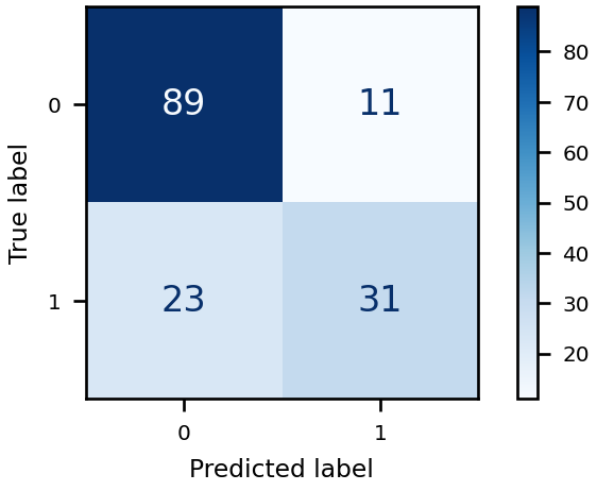


Figure 5. Confusion Matrix for The Baseline Model.

The confusion matrix Fig. 5 shows that the model correctly identified 89 true negatives and 31 true positives. However, it also made 23 false negatives and 11 false positives, which highlights the difficulty in identifying diabetic patients. The relatively higher number of false negatives suggests that the model was more likely to misclassify patients who had diabetes, leading to a lower recall.

3.2. Hyper-parameter Optimization

To improve the model, we explored multiple hyper-parameters, including learning rates, batch sizes, number of epochs, and early stopping patience. The best result was achieved with early stopping patience set to 20, which yielded a validation accuracy of 0.8312. This improvement in accuracy suggests that the model benefited from the extended training time provided by the longer patience value, allowing it to fine-tune the weights more effectively before stopping.

Key takeaways from this step were:

- The model's recall improved, capturing more true positive cases during validation.
- The higher validation accuracy indicated that the model had become better at distinguishing between diabetic and non-diabetic patients.

3.3. Test Set Evaluation

After finalizing the hyper-parameters, we retrained the model on the full training data and evaluated it on the unseen test set. However, the test accuracy dropped to 0.7078, indicating a performance decline when applied to new data. The key performance metrics on the test set are shown in Tab. 2. These results, especially the lower recall and F1 score, suggest that the model may have been over-fitting to the validation data [7]. While early stopping helped prevent over-fitting during optimization, the model struggled to generalize to the test set.

Metric	Score
Test Accuracy	0.7078
Balanced Accuracy	0.6685
Recall	0.5370
Precision	0.5918
AUC	0.8130
F1 Score	0.5631
F-beta Score (beta=0.1)	0.5912
F-beta Score (beta=10)	0.5375

Table 2. Performance Metrics on The Test Set.

The confusion matrix for the test set Fig. 6 illustrates the model's performance in classifying diabetic and non-diabetic patients. The model correctly identified 80 true negatives and 29 true positives. However, it made 25 false negatives and 20 false positives, reflecting its difficulty in correctly identifying diabetic patients, as indicated by the relatively low recall.

The model's relatively lower recall and significant number of false negatives, as shown in Fig. 6, further emphasize its struggles with diabetic patient identification. These results highlight the need for further regularization or alternative optimization strategies to improve the generalization performance.

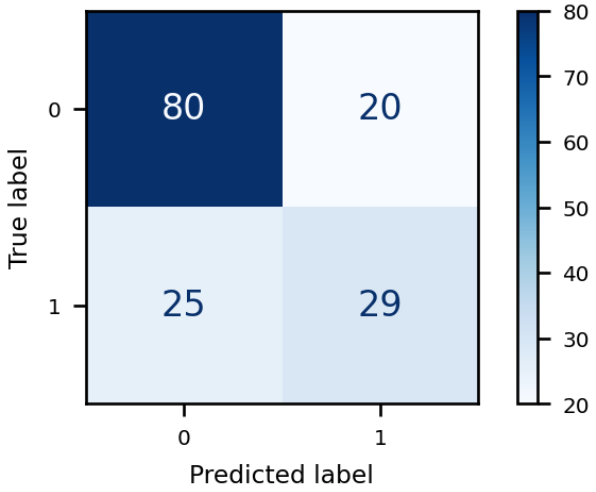


Figure 6. Confusion Matrix for The Test Set.

4. Code

Code is available at: https://github.com/ruhinb/Exploring-Perceptron-for-Diabetes-Prediction-A-Study-on-Model-Optimization

5. Conclusion

The results of these experiments indicate that while the hyper-parameter optimization improved the model's performance on the validation set, it did not fully translate to improved performance on the test set. This discrepancy underscores the importance of testing models on completely unseen data to ensure they generalize well in real-world scenarios. The model's precision remained strong, but the lower recall on the test set suggests that additional regularization or further hyper-parameter tuning may be necessary to balance the trade-offs between precision and recall.

Future work can focus on several enhancements to improve the current model's performance and generalization. First, exploring more complex architectures like Multilayer Perceptrons (MLPs) could help the model capture more intricate patterns in the data. Additionally, applying regularization techniques such as dropout and L2 regularization would reduce overfitting, a challenge observed during testing. Implementing k-fold cross-validation could offer a more comprehensive evaluation by testing the model on multiple subsets of data, improving reliability. Improvements in feature engineering and selection would allow for a more refined input set, potentially leading to better accuracy. Ensemble methods, such as bagging or boosting, could combine the strengths of different models to enhance predictive power. Finally, the performance could be significantly improved by incorporating more data, either through larger datasets or data augmentation. Together, these strategies offer promising avenues for boosting the model's effectiveness in real-world scenarios.

References

[1] Let's build a single layer perceptron (SLP) with python - meccanismo complesso. 3

[2] Pima indians diabetes database. 1, 2, 3

[3] Aman Darolia and Rajender Singh Chhillar. Analyzing three predictive algorithms for diabetes mellitus against the pima indians dataset. 107(1):2697–2704. 1

[4] Daniel H. Mathematical representation of a perceptron layer (with example in TensorFlow). 2

[5] Mariwan Hama Saeed. Diabetes type 2 classification using machine learning algorithms with up-sampling technique. 10:10. 1

[6] Md. Jamal Hossain, Md. Al-Mamun, and Md. Rabiul Islam. Diabetes mellitus, the fastest growing global public health concern: Early detection should be focused. 7(3):e2004. 1

[7] Osval Antonio Montesinos López, Abelardo Montesinos López, and Dr Jose Crossa. Overfitting, model tuning, and evaluation of prediction performance. In *Multivariate Statistical Machine Learning Methods for Genomic Prediction [Internet]*. Springer. 4

[8] Radhanath Patra and Bonomali Khuntia. Analysis and prediction of pima indian diabetes dataset using SDKNN classifier technique. 1070:012059. 1

[9] Giang Thu Vu, Bach Xuan Tran, Roger S. McIntyre, Hai Quang Pham, Hai Thanh Phan, Giang Hai Ha, Kenneth K. Gwee, Carl A. Latkin, Roger C.M. Ho, and Cyrus S.H. Ho. Modeling the research landscapes of artificial intelligence applications in diabetes (GAPRESEARCH). 17(6):1982. 1